

Lecture Notes: Deep Learning Basics and Optimization

1 Neural Network Basics

1.1 Fundamental Definitions

Let $z \sim \mathcal{N}(\mu, \Sigma)$. The probability density function is given by:

$$P(z) = \frac{1}{(2\pi)^{D/2} |\det \Sigma|^{1/2}} \exp \left(-\frac{1}{2} (z - \mu)^T \Sigma^{-1} (z - \mu) \right)$$

A basic neural network model function can be defined as:

$$y = \psi \left(\sum_i w_i x_i + b \right)$$

where w represents the weights, b represents the bias, and ψ is the activation function (e.g., $\frac{1}{1+e^{-x}}$, $\max(x, 0)$, $\tanh(x)$).

Using vector notation, we denote:

$$\sum_{i=1}^n w_i x_i + b = \sum_{i=0}^n w_i x_i =: \tilde{w}^T \tilde{x}$$

where $\tilde{w} = (b, w)$ and $\tilde{x} = (1, x_1, \dots, x_n)$.

The output of the j -th neuron in the k -th layer is:

$$y_{jk} = \psi \left(\sum_i w_{ji}^k y_{i,k-1} \right)$$

Here, w_{jk} is the weight vector of the j -th neuron in the k -th layer.

1.2 Matrix Formulation

Let $W_k = (w_{1k}, w_{2k}, \dots, w_{nk})^T$ represent the weights in the k -th layer. The output of the k -th layer can be expressed as:

$$y_k = \psi(W_k y_{k-1})$$

By composing these functions, we get:

$$y_k = \psi(W_k \psi(W_{k-1} \psi(W_{k-2} \dots \psi(W_2 \psi(W_1 x))))$$

A neural network can be summarized as a function:

$$y = g_\theta(x)$$

where θ represents the parameters.

2 Optimization Algorithms

To solve the task, we need to optimize an objective function $\max_\theta J(x; \theta)$. This requires an objective function J , a model g , and an optimization algorithm.

2.1 Basic Optimization

The gradient points in the direction where the function grows the fastest. By Taylor expansion:

$$f(x + \epsilon) = f(x) + \nabla f(x)^T \cdot \epsilon + O(\epsilon^2)$$
$$f(x + \epsilon) - f(x) \approx \nabla f(x)^T \cdot \epsilon = \|\nabla f(x)\| \|\epsilon\| \cos \theta$$

2.2 Gradient Method

The update rule for finding the maximum is:

$$w \leftarrow w + \mu \nabla f(w)$$

The update rule for finding the minimum is:

$$w \leftarrow w - \mu \nabla f(w)$$

Convergence can be monitored by checking conditions between iterations:

- $\|w_{t+1} - w_t\| < \epsilon$ (monitoring whether the change in parameters is small)
- $\|\nabla f(w)\| \approx 0$
- Change in the objective function is small

3 Choosing Step Size

Choosing the step size μ :

- If μ is too large, the algorithm will not converge at all.
- If μ is too small, the algorithm will be too slow.

To find the maximum of $f(w + \mu \nabla f(w))$ with respect to μ :

$$\max_{\mu} f(w + \mu \nabla f(w))$$

Compute the value of J using these three step sizes in the current step $\mu_0, \frac{\mu_0}{2}, 2\mu_0$, and choose the step size which gives the largest value for J . This depends on the initial point w_0 . Run from many initial points, compare the value of J in resulting final points, and choose the one with the highest value.

Matrix formula:

$$f(W) = g(WZ)$$
$$\nabla f(W) = g'(WZ) \cdot Z^T$$

(Note: W is a matrix)

$$W \leftarrow W + \mu g'(WZ) Z^T$$

4 Advanced Optimization Techniques

4.1 Constrained Optimization

Constrained optimization involves the projection of w back to the constrained set:

$$w \leftarrow \frac{w}{\|w\|}$$

(e.g., if the constrained set is the unit sphere).

4.2 Stochastic Gradient Descent (SGD)

For SGD, we don't need to compute the full gradient before taking a step. For each data point x_i :

$$w \leftarrow w + \mu \nabla_w g(w, x_i)$$

This is an approximation of the true gradient update:

$$w \leftarrow w + \mu \frac{1}{\text{size}(M)} \sum_{x \in M} \nabla_w g(w, x)$$

We don't need to calculate the gradient across the entire dataset for every iteration, which makes it faster. However, the step size must be smaller to compensate for randomness. In theory: $\sum \mu_t = \infty$ and $\sum \mu_t^2 < \infty$.

In practice, we use momentum by taking a moving average, keeping a record of recent gradients:

$$S \leftarrow (1 - \alpha)S + \alpha \nabla_w g(w_t, x_t)$$

4.3 Newton's Method

By Taylor expansion:

$$f(w) \approx f(w_0) + \nabla f(w_0)^T (w - w_0) + \frac{1}{2} (w - w_0)^T H_f(w_0) (w - w_0)$$

Setting the derivative to zero:

$$\frac{\partial f}{\partial w} = \nabla f(w_0) + H_f(w_0)(w - w_0) = 0$$

This leads to the update rule:

$$w \leftarrow w - (H_f(w))^{-1} \nabla f(w)$$

4.4 Conjugate Gradient (CG) Method

Algorithm:

1. $t = 0$, perform ordinary gradient method.
2. $t \leftarrow t + 1$
3. Define step direction: $\Delta w_t = -\nabla_w f(w_t)$
4. Compute β_t :

$$\beta_t^{FR} = \frac{\Delta w_t^T \cdot \Delta w_t}{\Delta w_{t-1}^T \cdot \Delta w_{t-1}}$$
$$\beta_t^{PR} = \frac{\Delta w_t^T (\Delta w_t - \Delta w_{t-1})}{\Delta w_{t-1}^T \cdot \Delta w_{t-1}}$$

5. Update conjugate direction: $S_t = \Delta w_t + \beta_{t-1} S_{t-1}$
6. Optimize: $\alpha_t = \text{argmax}_{\alpha} f(w_t + \alpha S_t)$
7. Update weights: $w_{t+1} = w_t + \alpha_t S_t$

5 Learning Neural Network Parameters

Reference: Prince book 7.2-7.3.

5.1 Forward and Backward Pass Example

Example: $f(x) = (x^2 + \sin(x^2)) \cdot x^2$ Let $a = x^2$, $b = \sin(a)$, $c = a + b$, and $f = ac$. The computational graph flows as: $x \rightarrow a \rightarrow b \rightarrow c \rightarrow f$

Forward Pass: $\frac{\partial a}{\partial x} = 2x$, $\frac{\partial b}{\partial a} = \cos(a)$, $\frac{\partial c}{\partial a} = 1$, $\frac{\partial c}{\partial b} = 1$, $\frac{\partial f}{\partial a} = c$, $\frac{\partial f}{\partial c} = a$

Backward Pass:

$$\begin{aligned}\frac{\partial f}{\partial c} &= a \\ \frac{\partial f}{\partial b} &= \frac{\partial f}{\partial c} \cdot \frac{\partial c}{\partial b} = a \\ \frac{\partial f}{\partial a} &= \frac{\partial f}{\partial c} \cdot \frac{\partial c}{\partial a} + \frac{\partial f}{\partial b} \cdot \frac{\partial b}{\partial a} \\ \frac{\partial f}{\partial x} &= \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial x}\end{aligned}$$

5.2 Backpropagation in Neural Networks

Forward pass for NN:

$$Z_1 = W_1 x \quad (\text{dimensions: } m \times n, n \times 1 \rightarrow m \times 1)$$

$$y_1 = \psi_1(Z_1)$$

Gradients with respect to intermediate variables:

$$\frac{\partial Z_1}{\partial W_1} = x^T, \quad \frac{\partial Z_1}{\partial x} = W_1^T, \quad \frac{\partial y_1}{\partial Z_1} = \psi'_1(Z_1)$$

Backward pass:

$$\begin{aligned}e_k &= \frac{\partial J}{\partial y_k} = 2(y_k - y) \\ \frac{\partial J}{\partial W_k} &= \frac{\partial J}{\partial y_k} \cdot \frac{\partial y_k}{\partial Z_k} \cdot \frac{\partial Z_k}{\partial W_k} = (e_k \odot \psi'(Z_k)) \cdot y_{k-1}^T \\ \frac{\partial J}{\partial y_{k-1}} &= \frac{\partial J}{\partial y_k} \cdot \frac{\partial y_k}{\partial Z_k} \cdot \frac{\partial Z_k}{\partial y_{k-1}} = (e_k \odot \psi'(Z_k)) \cdot W_k^T\end{aligned}$$

5.3 Adam Optimizer

Compute the gradient g_t .

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \cdot g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) \cdot g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ \theta_t &= \theta_{t-1} - \mu \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}\end{aligned}$$

Common hyperparameters: $\mu = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Definitions: Batch: B . N/B iterations $\Rightarrow$ 1 epoch. Techniques like residual connections and batch normalization are used. Dropout can be interpreted as training an ensemble of slightly different networks.

6 Generalization

Reference: Prince book 5.1-5.2.

For Maximum Likelihood Estimation (MLE):

$$\max J(W) = -\frac{1}{N} \sum_{n=1}^N \log p(y_n | \psi(Wx_n))$$

- **Regression:** $y \sim \mathcal{N}(Wx, \sigma^2)$, optimizing $-\frac{1}{2\sigma^2} \|y - Wx\|^2$

- **Classification:** $y \sim \text{Bern}(p)$, $p = \sigma(Wx) = \frac{1}{1+e^{-Wx}}$, optimizing the log-likelihood $-y \log(p) - (1-y) \log(1-p)$

Real Loss $\Rightarrow$ risk $R(\theta) = \mathbb{E}_{p(y,x)}[J(y, x, \theta)]$ **Training Loss** $\Rightarrow$ empirical risk $R_e(\theta) = \frac{1}{N} \sum_n J(y_n, x_n, \theta)$
 We want a low $R(\theta) = \frac{1}{N'} \sum_n J(y'_n, x'_n, \theta)$ evaluated by the model.

Bias: The difference/error introduced by approximating real-life problems.

Variance: The amount by which f would change if we estimate using different data.

6.1 Double Descent

For $D \gg N$ (where N is data size, D is model capacity), there are multiple solutions with perfect empirical risk $R_e(\theta)$. With suitable algorithms, we can find the ones that have better true risk $R(\theta)$. Classical view: More data always helps. Data paradox: For overparameterized models, let $\nu(N)$ denote the interpolation threshold. If $\nu(N_2) \gg \nu(N_1)$ and $D \gg \nu(N_1)$, $D \approx \nu(N_2)$, it might lead to $R(\theta_2) \gg R(\theta_1)$.

7 Regularization in Neural Networks

Reference: Prince Section 10.

- **Explicit Regularization:** L_2 and L_1 penalties (e.g., $\theta^T \theta$, $\sum |\theta|$).
- **Semi-implicit Regularization:** Early stopping. Stop the optimization process before the validation error increases. It needs averaging losses over multiple iterations or full-batch epoch losses to handle the stochasticity of updates.
- **Stochastic Gradient Descent (SGD):** Mini-batch updates have a regularizing effect, helping in finding robust solutions. The SGD objective can be formulated as:

$$J(\theta) + \frac{\mu}{4} \|\nabla J(\theta)\|^2 + \frac{\mu}{4B} \|\nabla J_b(\theta) - \nabla J(\theta)\|^2$$

where the latter term is the loss for the b -th mini-batch. SGD tends to find minima that are in some sense flat. The bias introduced by the choice of initialization, the trajectory of the optimization algorithm, and the constraints of the architecture are also important.

8 Supervised Learning and Beyond

8.1 Data Augmentation and Noise

- **Data Augmentation:** Multiply the size of the training data by creating new versions of original data (e.g., rotate, crop, scale, recolor). This increases the amount of data without changing the content.
- **Input and Output Noise:** Adding noise such as $\tilde{x} = x + \epsilon_x$ and $\tilde{y} = y + \epsilon_y$. From a data augmentation perspective, the noisy versions are treated as new, realistic samples.
- **Implicit Regularization:** Stochastic variations in gradients are generally good, suggesting that adding noise helps generalization.
- **Consistency Regularization:** Forcing methods to produce similar outputs for minor variations of inputs.

8.2 Ensembles and Dropout

- **Ensembles:** Output is averaged over multiple models: $h(x) = \frac{1}{M} \sum_{m=1}^M g_m(x)$.
- **Dropout:** Randomly remove some connections in each iteration. This can be interpreted as training an ensemble of slightly different networks, as Bayesian inference, or simply as adding some noise to the parameters.

8.3 Hyperparameters

- **Optimization:** Learning rate (μ), epochs, momentum, algorithm choice.
- **Generalization:** Weight decay (λ), noise variance, ensemble size, dropout probability.
- **Model:** Number of layers, units per layer, initialization strategy.

8.4 Advanced Learning Paradigms

- **Transfer Learning:** Keep the feature extraction part identical and only train the last layer to solve a new problem:

$$g(x) = \psi_k(W_k g_s(x))$$

where $g_s(x)$ is the source model without the output layer. Alternatively, continue training the feature extractor on the new data. Key concept: train on data sampled from $p_S(x, y)$ and evaluate on $p_T(x, y)$.

- **Few-shot Learning:** Built on nearest neighbor classification.
 1. Train a neural network on a large source dataset.
 2. Feed all test samples x through the network to obtain feature representations $f(x) \in \mathbb{R}^d$.
 3. Compute pairwise distances $d(f(x_i), f(x_j))$.
 4. Assign x_i with the label of its closest neighbor in the label set.
- **Multi-task Learning:** The objective is a weighted sum of losses for T tasks:

$$J(\theta) = \sum_{t=1}^T w_t J_t(\theta)$$

- **Zero-shot Learning:** Assume each class c has a feature vector z_c . Train a model $g(x)$ that outputs z_c for a sample belonging to that class.

9 Neural Network Architectures

Reference: Prince Section 10.

9.1 Fully Connected Layers

The basic architecture is naive:

$$g(x) = \psi_k(W_k(\psi_{k-1}(W_{k-1} \dots \psi_1(W_1 x))))$$

If we want lots of neurons (a large network) but fewer weights, we need:

- **Sparse Connectivity:** Only connect to a small subset of previous layer nodes (setting $w_{ij} = 0$ for many connections).
- **Parameter Sharing:** Use the same weight for multiple connections ($w_{ij} = w_{kl}$).

Examples of architectures utilizing these principles are CNNs and Transformers.

9.2 Convolutional Neural Networks (CNNs)

- **Convolution:** A basic operation can be defined as $z_i = w_1 x_{i-1} + w_2 x_i + w_3 x_{i+1}$. This relies on *locality* (the assumption that x_{i-1}, x_i, x_{i+1} have local relativity) and *parameter sharing* (the same weights W are used for all z_i).
- **Key Components:**
 - **Stride:** Used to reduce the spatial size.
 - **Pooling:** Max or average operations applied in a local neighborhood.
- **Pipeline:** The final spatial representation is flattened to a vector, which is then passed to a fully connected layer.

$$\text{Input} \rightarrow (\text{Stride and Pooling}) \times N \rightarrow \text{Flatten} \rightarrow \text{Fully Connected Layer}$$

9.3 Upsampling and Modern Architectures

- **Upsampling:** Common in encoder-decoder architectures. For example, an encoder maps $224 \times 224 \rightarrow 7 \times 7$, and a decoder maps $7 \times 7 \rightarrow 224 \times 224$.
- **Methods:** Explicit upsampling (interpolation) or transposed convolution, which can even reuse some weights.
- **Residual Connections (ResNet):** Replaces standard layer sequences with addition:

$$y_k = y_{k-1} + \psi(W_k y_{k-1})$$

- **DenseNet:** Uses concatenation instead of addition:

$$y_k = (y_{k-1}, \psi(W_k y_{k-1}))$$

10 Sequence Models and Attention Mechanisms

10.1 Recurrent Neural Networks (RNNs)

In feedforward networks, information flows only in one direction. In Recurrent Neural Networks (RNNs), the hidden state and output are updated as follows:

$$h_t = \psi_h(W_x x_t + W_h h_{t-1})$$

$$y = \psi_y(W_y h_t)$$

If we initialize $h_0 = 0$, the unfolding over time yields:

$$h_1 = \psi_h(W_x x_1)$$

$$h_2 = \psi_h(W_x x_2 + W_h h_1) = \psi_h(W_x x_2 + W_h \psi_h(W_x x_1))$$

$$h_3 = \psi_h(W_x x_3 + W_h h_2) = \psi_h(W_x x_3 + W_h \psi_h(W_x x_2 + W_h \psi_h(W_x x_1)))$$

RNNs appear to be overtaken by Transformers. However, variants like GRU are still notable:

- **Reset Gate:** When close to zero, it forces the network to forget the history.
- **Update Gate:** A linear combination of the previous hidden state and the candidate state.

10.2 Transformers and Attention

Reference: Prince Chapter 12.

Let $V_m = \beta_v + \Omega_v X_m$. The attention mechanism computes a weighted sum:

$$S_a[X_1, \dots, X_N] = \sum_{m=1}^N a[X_m, X_n] \cdot V_m$$

To compute the attention weights $a[X_m, X_n]$, we define queries and keys:

$$q_n = \beta_q + \Omega_q X_n$$

$$k_m = \beta_k + \Omega_k X_m$$

$$a[X_m, X_n] = \text{softmax}[k_m^T q_n] = \frac{\exp[k_m^T q_n]}{\sum_{m'=1}^N \exp[k_{m'}^T q_n]}$$

In Matrix Form:

$$V[X] = \beta_v \mathbf{1}^T + \Omega_v X$$

$$Q[X] = \beta_q \mathbf{1}^T + \Omega_q X$$

$$K[X] = \beta_k \mathbf{1}^T + \Omega_k X$$

$$S_a[X] = V[X] \cdot \text{softmax}[K[X]^T Q[X]]$$

With standard linear projections: $Q = W_Q X$, $K = W_K X$, $V = W_V X$. The self-attention output is:

$$Y = V \cdot \text{softmax}\left(\frac{Q^T K}{\sqrt{D}}\right)$$

10.3 Multi-Head Attention and Positional Encoding

For multi-head attention, we use M different parameter matrices W_V^m, W_Q^m, W_K^m .

1. Compute separately the attention weights A_{ij}^m and then output Y^m .
2. Concatenate the outputs: $Y = [Y^1, \dots, Y^M]$.
3. Pull back to original dimensionality with BY where $B \in \mathbb{R}^{D \times MD_v}$.

To encode position, let the position-augmented input be $\tilde{x}_i \triangleq x_i + p_i$. The dot product becomes:

$$y_{ij} = q_i^T k_j = (W_q(x_i + p_i))^T (W_k(x_j + p_j))$$
$$y_{ij} = x_i^T (W_q^T W_k) x_j + x_i^T (W_q^T W_k) p_j + p_i^T (W_q^T W_k) x_j + p_i^T (W_q^T W_k) p_j$$

The original Transformer paper used sinusoidal positional encodings:

$$p_{i,2j} = \sin\left(\frac{i}{C^{2j/D}}\right)$$
$$p_{i,2j+1} = \cos\left(\frac{i}{C^{2j/D}}\right)$$

where $C = 10000$.

10.4 Transformer Outputs and GPT-3

To tag each word with a word class (e.g., 'verb', 'adjective'), you can use a linear transformation and softmax with cross-entropy loss. Alternatively, a '[CLS]' token can be added to the sentence as a learnable vector to collect the whole sentence information.

GPT-3 is an autoregressive model estimating the joint probability:

$$p(x_1, \dots, x_N) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2) \dots p(x_N|x_1, \dots, x_{N-1})$$

This is achieved by killing connections to future tokens; we can only sum over the previous indices by setting $v_{ij} = -\infty$ for $j > i$ so that $A_{ij} = 0$.

11 Self-Supervised Learning and Autoencoders

11.1 Learning Paradigms

- **Supervised Learning:** Given input x and output y .
- **Unsupervised Learning:** Only input x is available.
- **Reinforcement Learning:** An agent that acts in a world based on x , takes actions a , and obtains a reward r .

11.2 Goals of Unsupervised Learning

1. Extract features for downstream supervised learning.
2. Find latent quantities and underlying structure (e.g., PCA, autoencoders).
3. Build an accurate model of data distribution.
4. Generate new sampling points.

11.3 Self-Supervised Learning (SSL)

In SSL, the neural network must discover the noise structure in the data. The supervised problem in SSL is called a *pretext task* (e.g., training a nonlinear classifier to discriminate observed data from some noise or shuffled real data).

- **Step 1:** Learn to predict the masked-out words (pretext task).
- **Step 2:** Train the model further for a specific supervised task.

When we do not have a lot of labeled data, the solution is to first train a NN via SSL to learn features using a large unlabeled dataset, and then use the learned features as input to a simple classifier with a small labeled dataset.

Foundation models are typically trained on broad data via SSL with billions of parameters.

11.4 Autoencoders and PCA

The basic autoencoder predicts x from x . This alone yields useless knowledge unless constrained.

- **Case 1 (Bottleneck):** A hidden layer with fewer units forces the network to summarize.
- **Case 2 (Sparsity):** Adding a penalty term to the loss function.
- **Case 3 (Denoising):** Corrupt the input (e.g., add Gaussian noise, remove input variables, remove color) and train the network to reconstruct the original input.

Principal Component Analysis (PCA) can be viewed as a linear autoencoder.

- **Encoder:** $S = Wx$, where $W \in \mathbb{R}^{m \times n}$ and $m = \dim(S) < n$.
- **Decoder:** $\hat{x} = AS$, so $\hat{x} = AWx$.

The objective is to minimize $\mathbb{E}\|x - \hat{x}\|^2$, which has an equivalent definition of maximizing $\mathbb{E}(W^T x)^2$ subject to $\|W\| = 1$.

12 Downstream Tasks and Autoencoders

12.1 Downstream Tasks and Self-Supervised Learning

A downstream task involves training a simple classifier with a small labeled dataset using features extracted by a model during a “pretext” or “upstream” phase.

Self-Supervised Learning (SSL) is one technique for unsupervised learning that uses supervised learning algorithms to achieve unsupervised learning. Note that not all unsupervised learning is SSL.

12.2 Autoencoders

Reference: Bishop’s book Section 10.1, 19.1.

The basic idea of a dimension-reducing autoencoder is to predict x by x . The data flows from the input x to a hidden layer Z (with a small number of units) to the output $\hat{x}$.

Definition of a dimension-reducing autoencoder: For n -dimensional input data x :

- **Encoding:** m -dimensional representation $S = g_\theta(x)$, where $g_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^m$.
- **Decoding:** $\hat{x} = h_\eta(S)$, where $h_\eta : \mathbb{R}^m \rightarrow \mathbb{R}^n$.

The objective function is to minimize the reconstruction error with respect to both θ and η :

$$\min_{\theta, \eta} \mathbb{E}\|x - \hat{x}\|^2 = \mathbb{E}\|x - h_\eta(g_\theta(x))\|^2$$

Linear autoencoders function similarly to linear PCA. We can generalize linear PCA to nonlinear PCA.

12.3 Definition of a Manifold

A manifold is a generalization of the concept of a subspace to the nonlinear case.

- **Definition 1:** A lower-dimensional (m) smooth set in a high-dimensional space (n) defined by a smooth function $f : U \rightarrow \mathbb{R}^n$ for a set $U \subseteq \mathbb{R}^m$.
- **Definition 2:** The set of points $\xi \in \mathbb{R}^n$ for which $g(\xi) = 0$, given a smooth function $g : \mathbb{R}^n \rightarrow \mathbb{R}^{n-m}$.

Real data is concentrated near a manifold, which can be expressed conceptually as:

$$\text{Data} = \text{Manifold} + \text{Noise}$$

13 Identifiability and Penalized Autoencoders

13.1 Lack of Identifiability

Linear Case:

$$\mathbb{E}||x - \hat{x}||^2 = \mathbb{E}||x - AWx||^2 \quad \text{where } V = AW$$

If we introduce an invertible matrix M :

$$A' = AM^{-1}, \quad W' = MW$$

$$A'W' = AM^{-1}MW = AW$$

The components $S = Wx$ are only defined up to a linear combination transformation. Therefore, there is no point in trying to interpret the individual components s_i . The columns of A and the columns of A' span the same subspace:

$$\text{Col}(AM^{-1}) \subseteq \text{Col}(A) \quad (\text{linear combinations of } A)$$

$$A = (AM^{-1})M \implies \text{Col}(A) \subseteq \text{Col}(AM^{-1}) \quad (\text{linear combinations of } AM^{-1})$$

Nonlinear Case:

$$\mathbb{E}||x - \hat{x}||^2 = \mathbb{E}||x - h(g(x))||^2$$

If we introduce an invertible nonlinear function k :

$$h'(S) = h(k(S)), \quad g'(x) = k^{-1}(g(x))$$

$$h'(g'(x)) = h(k(k^{-1}(g(x)))) = h(g(x))$$

13.2 Penalized Autoencoders

A penalty function is added to the objective function to constrain the representations:

$$\sum_{i=1}^N ||x^{(i)} - h(g(x^{(i)}))||^2 + \lambda \sum_j |s_j|$$

This formulation leads to concepts like **Sparse Autoencoders** and **Denoising Autoencoders**.

Denoising Autoencoders: This is a form of self-supervised learning achieved by adding noise and then denoising.

1. Add noise: $\tilde{x} = x + \text{noise}$.
2. Train a neural network to learn a denoising function to reconstruct x from $\tilde{x}$.

The objective function is:

$$J(\theta) = \sum_{i=1}^N ||x^{(i)} - g_{\theta}(\tilde{x}^{(i)})||^2$$

14 Energy-Based Models (EBM) and Score Matching

14.1 Energy-Based Models

Assume observed data $x \sim p(x; \theta)$. We observe a sample of X (N observations) from $p(x; \theta^*)$ and aim to estimate θ . In classical methods, we use Maximum Likelihood Estimation (MLE) to find $\hat{\theta}$.

In EBMs, we define the model probability density function as:

$$p_{\text{model}}(x; \theta) = \frac{1}{Z(\theta)} p_{\text{un}}(x; \theta)$$

where $Z(\theta)$ is the unknown partition function:

$$Z(\theta) = \int_{\mathbb{R}^n} p_{\text{un}}(\xi; \theta) d\xi$$

Model Parameterization: We want to use a neural network as a generative model of the data PDF. The unnormalized probability $p_{\text{un}}(x; \theta)$ is given by the output of a neural network with a scalar output, where θ represents the network parameters.

14.2 Score Matching

We define the model score function $\phi : \mathbb{R}^n \rightarrow \mathbb{R}^n$:

$$\phi(x; \theta) = \nabla_x \log p_{\text{model}}(x; \theta) = \nabla_x \log p_{\text{un}}(x; \theta) - \nabla_x \log Z(\theta)$$

Since $\nabla_x \log Z(\theta) = 0$, we have:

$$\phi(x; \theta) = \nabla_x \log p_{\text{un}}(x; \theta)$$

We also define the data score function (the score of the observed data):

$$\phi_x(x) = \nabla_x \log p_x(x)$$

The score matching objective function is defined as:

$$J(\theta) = \frac{1}{2} \int_{\mathbb{R}^n} p_x(\xi) \|\phi(\xi; \theta) - \phi_x(\xi)\|^2 d\xi$$

We want to find $\hat{\theta} = \arg \min_{\theta} J(\theta)$, but we do not know the true data distribution p_x .

Theorem: Assume some regularity conditions and smooth densities. Then the score matching objective function J can be expressed as:

$$J(\theta) = \int_{\mathbb{R}^n} p_x(\xi) \sum_{j=1}^n \left[\partial_j \phi_j(\xi; \theta) + \frac{1}{2} \phi_j(\xi; \theta)^2 \right] d\xi + \text{const.}$$

where $\phi_j(\xi; \theta) = \frac{\partial \log p_{\text{un}}(\xi; \theta)}{\partial \xi_j}$. This formulation successfully gets rid of the explicit dependence on the unknown data score $\phi_x(x)$.

Proof Sketch: Expanding the objective function:

$$J(\theta) = \frac{1}{2} \int p_x(\xi) \|\phi(\xi; \theta)\|^2 d\xi - \int p_x(\xi) \phi_x(\xi)^T \phi(\xi; \theta) d\xi + \text{const.}$$

Using integration by parts on the cross-term:

$$\begin{aligned} \int p_x(x) (\nabla_x \log p_x(x))^T \phi(x; \theta) dx &= \int p_x(x) \frac{\nabla_x p_x(x)}{p_x(x)} \phi(x; \theta) dx \\ &= \int (\nabla_x p_x(x)) \phi(x; \theta) dx = 0 - \int p_x(x) \nabla_x \cdot \phi(x; \theta) dx \end{aligned}$$

We can replace the integration (expectation) over p_x with a sample average:

$$\tilde{J}(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^n \left[\partial_j \phi_j(x^{(i)}; \theta) + \frac{1}{2} \phi_j(x^{(i)}; \theta)^2 \right]$$

14.3 Example: Gaussian Distribution

Let $p(x; M) = \frac{1}{Z(M)} \exp\left(-\frac{1}{2}x^T M x\right)$, where M is the inverse of the covariance matrix.

$$\phi(x; M) = \nabla_x \log p_{un}(x; M) = -Mx$$

$$\partial_j \phi_j(x; M) = -m_{jj}$$

$$\tilde{J}(M) = -\sum_{k=1}^n m_{kk} + \frac{1}{2N} \sum_{i=1}^N (Mx^{(i)})^T (Mx^{(i)})$$

In 1D context, where $M = \frac{1}{\sigma^2}$:

$$\tilde{J}(\sigma^2) = -\frac{1}{\sigma^2} + \frac{1}{2\sigma^4} \frac{\sum x_i^2}{N}$$

14.4 Denoising Score Matching (DSM)

We add a small Gaussian noise to the data: $\tilde{x} = x + n$. The objective is formulated as:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \|x^{(i)} - (\tilde{x}^{(i)} + \sigma^2 \phi(\tilde{x}^{(i)}; \theta))\|^2$$

where σ^2 is the variance of the added noise.

We are essentially fitting the expected value $\mathbb{E}[x|\tilde{x}]$ using least squares regression. Theoretically:

$$\mathbb{E}[x|\tilde{x}] \approx \tilde{x} + \sigma^2 \nabla_{\tilde{x}} \log p_x(\tilde{x})$$

The neural network learns a function $f(\tilde{x}) \approx \tilde{x} + \sigma^2 \phi(\tilde{x}; \theta)$, which matches the theoretical result governed by the score of the noisy distribution.

15 Noise-Contrastive Estimation (NCE)

15.1 Basic Formulation

Given an observed dataset $X = (x^{(1)}, \dots, x^{(N)})$ with an unknown probability density function (pdf) p_x , and noise $Y = (y^{(1)}, \dots, y^{(N)})$ with a known pdf p_y .

A neural network model $g(u; \theta)$ is used to model the data $\log p_x(u)$, where g corresponds to the unnormalized log-probability $\log p_{un}$.

$$G(z; \theta) = g(z; \theta) - \log p_y(z)$$

Using logistic regression to model the log-odds:

$$r(u; \theta) \approx \log \frac{P(C=1|Z=u)}{1 - P(C=1|Z=u)}, \quad C \in \{1, 2\}$$

$$P(C=1|Z=u) = \sigma(r(u; \theta))$$

where the sigmoid function is $\sigma(r) = \frac{1}{1 + \exp(-r)}$.

The log-likelihood objective for NCE is:

$$J(\theta) = \sum_{i=1}^N \left(\log[\sigma(r(x^{(i)}; \theta))] + \log[1 - \sigma(r(y^{(i)}; \theta))] \right)$$

15.2 Derivation of the Log-Odds

In basic logistic regression, $P(y = 1|X) = \sigma(z) = \frac{1}{1+e^{-z}}$, where $z = w^T x + b$. The derivation starts from the log-odds $\ln\left(\frac{p}{1-p}\right) = r(u; \theta)$:

$$P(C = 1|u) = \frac{p(u|C = 1)P(C = 1)}{p(u|C = 1)P(C = 1) + p(u|C = 2)P(C = 2)} = \frac{p_x(u)}{p_x(u) + p_y(u)}$$

$$P(C = 1|u) = \frac{1}{1 + \exp(-r(u; \theta))}$$

Therefore, $r(u; \theta) \rightarrow \ln p_x(u) - \ln p_y(u)$. We define:

$$g(u; \theta) = \log p_x(u)$$

$$G(u; \theta) = \log p_x(u) - \log p_y(u) = g(u; \theta) - \log p_y(u)$$

For an energy-based model, $p(x; \theta) = \frac{p_{un}(x; \theta)}{Z(\theta)}$, which implies $\ln p(x; \theta) = \ln p_{un}(x; \theta) - \ln Z(\theta)$.

15.3 Sampling from Energy-Based Models (EBM)

Reference: Bishop's book.

- **The choice of noise:** It should be easy to sample, have a properly normalized expression, and yield a small mean-squared error for the estimator.
- **Idea:** Sampling generates new images, which is the basis of Generative AI.
- **MCMC method:** An iterative process producing a sequence of samples $\xi_1, \xi_2, \dots$

16 Langevin Algorithm and Diffusion Models

16.1 Langevin Algorithm

The Langevin dynamics update rule is:

$$x_{t+1} = x_t + \mu \phi_x(x_t) + \sqrt{2\mu} n_t$$

where:

- $\phi_x = \nabla_x \log p(x)$ is the score function of x .
- μ is the step size.
- $n_t \sim \mathcal{N}(0, I)$ is the noise.
- $\mu \phi_x(x_t)$ gives the direction of increasing probability.
- n_t introduces randomness.

Example: Let $\tilde{x}^{(i)} = x^{(i)} + n^{(i)}$. To denoise, the objective is:

$$J(\theta) = \sum_{i=1}^N \left\| x^{(i)} - \left[\tilde{x}^{(i)} + \sigma^2 \phi(\tilde{x}^{(i)}; \theta) \right] \right\|^2$$

And the update step becomes $x_{t+1} = x_t + \mu \phi(x_t; \theta^*) + \sqrt{2\mu} n_t$.

Denoising Score Matching (DSM) does not estimate the real score function, but a noisy version. Solutions to this include:

1. Using a small noise level (which can be unstable).
2. Estimating the score function for several noise levels.
3. Utilizing Diffusion models.

16.2 Generative Diffusion Models

Diffusion models learn $p(x|u)$, where u is some representation of a prompt. The conditional score function is $\phi_{x|u}(x|u) = \nabla_x \log p(x|u)$. For example, if $u = 1$ represents a cat and $u = 2$ represents a dog, the score of the desired class is used for generation.

The core idea is moving from a structured case towards randomness. It is defined as a stochastic process: starting from an initial distribution P , Gaussian noise is added little by little until it ends up as pure Gaussian noise. Generative AI reverses this diffusion process using a denoising diffusion model (which acts as a denoising autoencoder).

The forward diffusion process is:

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \eta_t$$

To reverse this, we try to find $p(x_{t-1}|x_t)$ by learning a neural network $g_\theta(x; t)$ to estimate $\mathbb{E}[x_{t-1}|x_t]$ (via DAE or DSM). We also approximate the variance $\text{Var}[x_{t-1}|x_t]$ as a function of β_t . The denoising network $g_\theta(x; t)$ must be learned for many noise levels (time steps).

16.3 Sampling from a Generative Diffusion Model

Input: Denoising network $g(z, w, t)$, noise schedule $\beta_1, \dots, \beta_T$. **Output:** Sample vector x in the data space.

1. Sample $z_T \sim \mathcal{N}(0, I)$.
2. For $t = T, \dots, 2$:

$$\alpha_t = \prod_{\tau=1}^t (1 - \beta_\tau)$$

$$\mu(z_t, w, t) \leftarrow \frac{1}{\sqrt{1 - \beta_t}} \left\{ z_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} g(z_t, w, t) \right\}$$

Sample $\varepsilon \sim \mathcal{N}(0, I)$ from a noise vector.

$$z_{t-1} \leftarrow \mu(z_t, w, t) + \sqrt{\beta_t} \varepsilon$$

3. Final step:

$$x = \frac{1}{\sqrt{1 - \beta_1}} \left[z_1 - \frac{\beta_1}{\sqrt{1 - \alpha_1}} g(z_1, w, 1) \right]$$

4. Return x .

Training objective for the denoising network:

$$\min_{\theta} \|\varepsilon - g(\sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \varepsilon, w, t)\|^2$$

17 Deep Generative Models

17.1 Latent Variable Models

In latent variable models, x is defined as a function of a latent vector z .

- z is sampled from a prior distribution $p(z)$.
- x is sampled from $p(x|z)$.
- The joint distribution is $p_\theta(x, z) = p_\theta(x|z)p(z)$, where θ is a vector of parameters.

For zero-mean data, the covariance is $\text{cov}(z) = \mathbb{E}(zz^T)$. Applying a transformation M yields $\text{cov}(Mz) = M\text{cov}(z)M^T$. If $\text{cov}(z) = I$, it has whiteness properties.

Basic Instance: Let $p(z) \sim \mathcal{N}(0, I)$ and $x = f_\theta(z) + n$, where $n \sim \mathcal{N}(0, \frac{1}{\sigma^2} I)$ and f_θ is a nonlinear function. Here, $\dim(x)$ can be larger than $\dim(z)$ (dimension reduction). Without noise, the density $p(x)$ may not exist.

Estimation Challenges: The marginal likelihood is $p_\theta(x) = \int p_\theta(x, z) dz$. Maximizing the log-likelihood is difficult:

$$\log L(\theta; x_1, \dots, x_N) = \sum_{i=1}^N \log p_\theta(x_i)$$

Similarly, the Maximum a Posteriori (MAP) approximation:

$$\max_{\theta, z_1, \dots, z_N} \sum_{i=1}^N \log p_\theta(x_i | z_i) + \log p(z_i)$$

is computationally intractable. For the specific example:

$$p_\theta(x|z) = \frac{1}{(2\pi)^{d/2} \sigma^d} \exp\left(-\frac{1}{2\sigma^2} \|x - f_\theta(z)\|^2\right)$$

$$p_z(z) = \frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{1}{2} \|z\|^2\right)$$

17.2 Variational Autoencoders (VAE)

VAEs approximate the intractable posterior likelihood $p(z|x)$ using a variational distribution $q_\alpha(z|x)$. The objective is to maximize the evidence lower bound (ELBO):

$$J(\theta, \alpha) = \max_{\alpha, \theta} \sum_{i=1}^N \int q_\alpha(z|x^{(i)}) \log \frac{p_\theta(x^{(i)}, z)}{q_\alpha(z|x^{(i)})} dz$$

We want to obtain q_α to approximate the real distribution $p(z|x)$ to extract the latent representations z .

17.3 Generative Adversarial Networks (GAN)

GANs adjust the parameters of the latent variable models so that the distribution of generated data is as close as possible to the distribution of observed data.

1. Generate data from the current model.
2. Train a neural network discriminator to distinguish between generated data and real data.
3. Consider the classification error as a measure of distance between the two distributions.
4. Reduce the distance using gradient descent on the generator's parameters θ .

The optimization is formulated as a minimax game:

$$\min_{\theta} \max_{\alpha} -J_{err}(X, Y(\theta); \alpha)$$

Updates are performed via $\alpha \leftarrow \alpha - \mu_\alpha \nabla_\alpha(\dots)$ for the discriminator and $\theta \leftarrow \theta + \mu_\theta \nabla_\theta(\dots)$ for the generator.

18 Permutation Contrastive Learning (PCL)

1. Define short time windows as new data:

$$y(t) = (x(t), x(t-1))$$

2. Create randomly time-permuted data:

$$y^*(t) = (x(t), x(t^*))$$

where t^* is a random time point.

3. Train a neural network to discriminate the true sequence y from the permuted sequence y^* .

This approach performs nonlinear Independent Component Analysis (ICA) for time-dependent data.

18.1 Comparison: GAN vs. PCL

- **GAN (Generative Adversarial Networks):**

1. Relies on sampling new data.
2. Formulated as a min-max optimization problem.

- **PCL (Permutation Contrastive Learning):**

1. Provides theoretical identifiability (the model is strictly identifiable).
2. Framed as a Self-Supervised Learning (SSL) classification task.